



# Software engineering

## Authors

Matt ter Steege  
Matttersteege@gmail.com

## Contents

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>Software requirements . . . . .</b> | <b>2</b> |
| 1.1      | Wat zijn requirements? . . . . .       | 2        |
| 1.2      | Hoe schrijf je requirements . . . . .  | 2        |
| 1.3      | Requirements Engineering . . . . .     | 2        |
| 1.4      | Use Cases . . . . .                    | 3        |
| 1.5      | User Stories . . . . .                 | 3        |
| 1.6      | Quick Prototype . . . . .              | 3        |
| 1.7      | A/B Testing . . . . .                  | 3        |
| <b>2</b> | <b>UML . . . . .</b>                   | <b>3</b> |
| 2.1      | Relations . . . . .                    | 3        |

---

# 1 | Software requirements

## Wat zijn requirements?

*“Requirements define what a system should do and under what constraints it should operate”*

Als je een systeem ontwikkeld, dan hebben de gebruikers vaak eisen waaraan je wilt voldoen. Requirements engineering bevordert de communicatie tussen stakeholders<sup>1</sup> en geeft het eindproduct dus meer sturing. Het kost wel geld...

Requirements moeten: **Correct, Precise, Complete, Consistent & Verifiable** zijn (alternatief is SMART). Er zijn 2 soorten requirements:

### Functionele requirements

Wat moet een systeem doen

### Non-Functionele requirements

Onder welke condities, vaak met metrieke. Voorbeelden hiervan zijn bijvoorbeeld transacties per seconden, laadtijd, opslaggebruik, RAM gebruik, uptime percentage etc. dingen die wel te meten zijn, maar niet functioneel zijn.

## Hoe schrijf je requirements

Requirements moeten aan een aantal eigenschappen voldoen:

- **Correct** – beschrijft de juiste operaties en berekeningen.
- **Precise** – er is geen ambiguïteit in de beschrijving.
- **Complete** – alle noodzakelijke functionaliteiten zijn beschreven.
- **Consistent** – alle stakeholders hebben dezelfde interpretatie van de specificatie.
- **Verifiable** – het is mogelijk om te controleren of de implementatie aan de requirement voldoet.

Een alternatief hiervoor is SMART: Specific, Measurable, Achievable, Relevant, Time-bound.

Voorbeeld (**slechte requirements**):

---

“The system should be usable by elderly people.” Dit is niet goed verifieerbaar en “elderly” is onduidelijk.

“The product shall respond rapidly.” “Rapidly” is niet concreet en dus niet verifieerbaar.

Beter: “The system shall respond in less than 2s in 99% of the cases.”

---

## Requirements Engineering

**Requirements Engineering** is het identificeren, analyseren, specificeren en valideren van de requirements van een systeem. Dit gebeurt in overleg tussen developers en stakeholders. Een stakeholder is een persoon, groep of entiteit die een belang heeft bij of zich zorgen maakt over de realisatie van het systeem.

### Requirements verzamelen

Requirements kunnen op verschillende manieren worden verzameld:

- Praten met stakeholders (interviews, brainstorm, vragenlijsten)
- Bestaande documenten en systemen analyseren.
- Observeren hoe toekomstige gebruikers momenteel werken.
- Prototyping.
- Task analysis: onderzoeken welke subtaken er zijn en hoe deze in een hiërarchie kunnen worden georganiseerd.
- Scenario-based analysis: onderzoeken hoe een taak wordt uitgevoerd en wat er gebeurt in uitzonderingssituaties.

---

<sup>1</sup>Een persoon, groep of entiteit die te maken heeft met het maken van het product

---

## Use Cases

Een Use Case is een lijst van acties of gebeurtenissen die de interactie beschrijft tussen een rol (*actor*) en een systeem om een bepaald doel te bereiken. De actor kan een mens of een ander extern systeem zijn. Use Cases zijn vooral geschikt voor het identificeren van **functionele requirements op hoog niveau**. Ze worden beschreven in gestructureerde tekst en zijn gedetailleerder dan user stories.

## User Stories

User stories zijn een lightweight aanpak voor requirements en use-case-analyse. Ze beschrijven hoe een applicatie naar verwachting gebruikt wordt en helpen stakeholders bij het plannen en prioriteren van de ontwikkeling. Een user story bevat drie onderdelen, ook wel de 3 R's genoemd:

- **Role** – wie wil iets?
- **Requirement** – wat wil die persoon?
- **Reason** – waarom wil die persoon dit?

De standaardvorm is: “As a *[role]*, I want *[requirement]*, so that *[reason]*.”

Voorbeeld (**User story**):

---

As a new user, I want to sign up using my existing Google account so that I don't have to keep track of another account.

Een functionele requirement:

The system must allow users to search for songs, albums, and artists using keywords.

kan bijvoorbeeld worden herschreven als:

As a user, I want to search for songs, albums, and artists using keywords, so that I can easily find and listen to the music I enjoy.

---

## Quick Prototype

Een andere manier om requirements te ontwikkelen is een **Quick Prototype**. Hierbij werk je nauw en continu samen met stakeholders om requirements en tests te ontwikkelen. Er wordt een werkend prototype onderhouden terwijl nieuwe features worden toegevoegd. Dit gebeurt typisch in korte iteraties, bijvoorbeeld iedere twee weken. Na iedere iteratie wordt met stakeholders besproken wat er als volgende moet worden toegevoegd. Hierdoor kan worden gecontroleerd of het huidige systeem daadwerkelijk overeenkomt met wat stakeholders willen.

## Minimum Viable Product (MVP)

Een Minimum Viable Product (MVP) is een subset van de volledige verzameling features die, wanneer deze af is, al *business value in de echte wereld* heeft. Het doel is dus niet om meteen het volledige systeem te bouwen, maar om een minimale versie te maken die al waarde heeft.

## A/B Testing

Bij A/B testing worden minimaal twee versies van een systeem aan verschillende groepen gebruikers getoond. Elke groep krijgt een andere versie te zien. Bijvoorbeeld:

- Versie A implementeert requirement A.
- Versie B implementeert requirement B.

De requirements zijn hierbij onderling exclusief. Door de verschillende versies aan gebruikers te tonen, kan worden onderzocht welke variant het beste aansluit bij het gewenste gebruik.

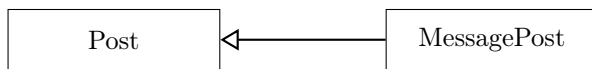
# 2 | UML

## Relations

Lees dit als: **MessagePost is a Post**

## Inheritance / Generalization

Inheritance means that one class is a specialized version of another: an **is-a** relationship.

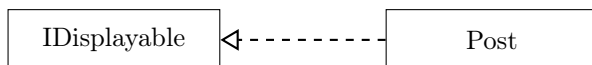


Lees dit als: **MessagePost is a Post**

De lege driehoek wijst naar de superclass, dus Subclass → Superclass. Het voorbeeld uit het hoorcollege is dat “MessagePost is derived from Post”.

## Realization

Realization is used when a class **implements an interface**.



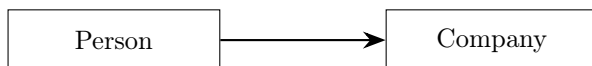
Lees dit als: **Post implements IDisplayable**  
het verschil met Inheritance is:

|                         |               |
|-------------------------|---------------|
| solid + empty triangle  | = inheritance |
| dashed + empty triangle | = realization |

De driehoek wijst naar de interface die geïmplementeerd wordt.

## Association

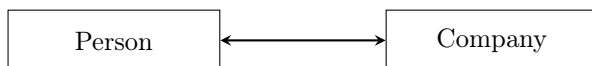
Een associatie houdt in dat klassen met elkaar verbonden zijn, doorgaans doordat de ene klasse **weet van** de andere.



Lees dit als: **Person knows Company**

De pijl wijst naar de class waarover geweten wordt (lekker genederlandst). Knower → Known class.

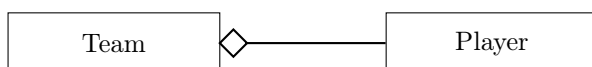
Een pijl die beide richtingen opgaat (bidirectionaal) betekend dat beide klassen van elkaar weten:



Het voorbeeld was dat een persoon weet bij welk bedrijf hij werkt, en het bedrijf weet dat hij bij hun werkt.

## Aggregation and Composition

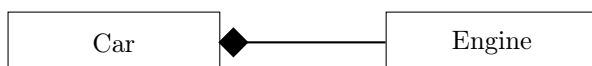
Beide beschrijven een **part-of** / **has-a** relatie. Het verschil zit in de sterkte van het eigendom.



**Aggregation** gebruikt een lege diamant. De diamant is aan de kan van de “container” kant.

Lees dit als: **Team has Players**

Het belangrijke is dat een deel kan bestaan zonder de ander, een team bestaat ook wel zonder een bepaalde speler, en vice versa.



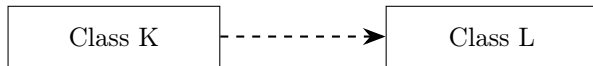
**Composition** gebruikt een gevulde diamant. De diamant is aan de kan van de “container” kant. Compositie is een **sterke vorm van aggregatie**. Het ingesloten object maakt deel uit van het omvattende object en heeft geen zelfstandig bestaansrecht.

Lees dit als: **Car requires engine & engine requires a car**

Voor het examen: Diamond always goes on the “whole” side.

## Dependency

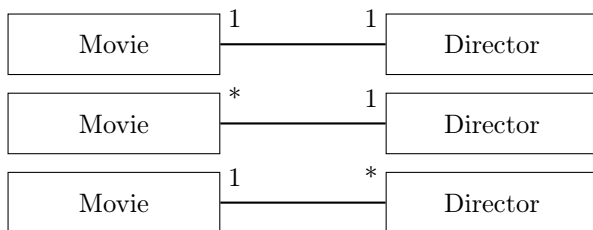
Een dependency houdt in dat de ene klasse een andere klasse **gebruikt of daarvan afhankelijk is**. Het college noemt ook het creëren van een andere klasse als voorbeeld.



Lees dit als: **K depends on L**

De pijl wijst naar de class die gebruikt wordt. Een belangrijk onderscheid is dat een afhankelijkheid **niet** noodzakelijkerwijs betekent dat *K* een *L*-object als attribuut opslaat. Meer in het algemeen geldt: als een wijziging in *L* gevolgen kan hebben voor *K*, dan is *K* afhankelijk van *L*.

## Multiplicity / Cardinality



Multiplicity geeft aan **hoeveel objecten** aan een relatie kunnen deelnemen. Veer voorkomende voorbeelden zijn:

|      |              |
|------|--------------|
| 1    | exactly one  |
| 0..1 | zero or one  |
| *    | zero or many |
| 1..* | one or many  |

Het voorbeeld hiernaast zegt:

- Elke film heeft precies één regisseur.
- Elke regisseur heeft 0 of meer films geregisseerd.
- Elke film kan meerdere regisseurs hebben.